

FLASH-RF Configuration Performance Tuning: A Seven-Iteration Study from Random Search to RL-Driven Strategy

0. Research Overview	3
1. Problem Definition & Dataset	4
1.1 Problem Formalisation	4
1.2 Key Problem Characteristics	4
1.3 Dataset	4
1.4 Dataset Processing	5
1.5 Experimental Evaluation Framework	5
2. Algorithm Survey & Selection	5
2.1 Survey Scope	5
2.2 Surveyed Algorithm Families	6
2.3 Selection Rationale	7
2.4 Why CART over Gaussian Processes?	8
2.5 Limitations	9
3. Iteration 1: v1 — FLASH Framework + RF Ensemble	9
3.1 Starting Point & Motivation	9
3.2 Algorithm Design	9
3.3 Main Experimental Results	10
3.4 TPE Controlled Experiment	11
3.5 Result Analysis	11
3.6 Root-Cause Diagnosis of the 7z Failure	11
3.7 v1 Hyperparameter Sensitivity Analysis	12
3.8 Remaining Issues & Next Steps	13
4. Iteration 2: v2 — RF-LCB + Feature Preprocessing	14
4.1 What Changed	14
4.2 Algorithm Design	14
4.3 Dataset Processing Changes	14
4.4 Experimental Design & Execution	15
4.5 Experimental Results	15
4.6 Result Analysis	15
4.7 Remaining Issues	16
5. Iteration 3: v3 — LightGBM Attempt (Rejected)	16

5.1 Motivation: Can Boosting Beat Bagging?	16
5.2 Why LightGBM?	17
5.3 v3.0: Quantile Regression — Complete Failure	17
5.4 v3.1: Bagging of Boostings	17
5.5 Why Boosting Lost	18
5.6 Takeaway	18
6. Iteration 4: v4 — Thompson Sampling	18
6.1 Why Thompson Sampling?	18
6.2 Algorithm Design	19
6.3 Experimental Results	19
6.4 Result Analysis	19
6.5 v3 vs v4 — Model vs Strategy	20
7. Iteration 5: v5 — Conditional Log Transform	20
7.1 Gap Analysis	20
7.2 Diagnosis	21
7.3 What I Tried	21
7.4 Results	22
7.5 Remaining Blind Spot	23
8. Iteration 6: v6 — Conditional Stratified Initial Sampling	23
8.1 Motivation	23
8.2 Algorithm Design	23
8.3 Rejected Alternatives	24
8.4 Experimental Results	24
8.5 Result Analysis	24
9. Iteration 7: v7 — Rank Transform (Rejected)	24
9.1 Motivation	24
9.2 Results	25
9.3 Why It Failed	25
10. Final Validation: Multi-Method Multi-Budget Comparison	26
10.1 6-Method Full Comparison (R=100)	26
10.2 Multi-Budget Scenarios (R=30/100/200)	26
11. Global Review	27
11.1 Version Comparison	27
11.2 Iteration Logic	28
12. Conclusion	28
Appendix A: Per-Family Algorithm Survey Details	29
A.1 Baseline Methods (RS, LHS, Sobol, grid search)	29
A.2 Trajectory-Based (HC, SA, Tabu, VNS)	29

A.3 Population-Based (GA, DE, PSO, CMA-ES, BestConfig, irace)	30
A.4 Surrogate/Model-Based (GP-BO, SMAC, TPE, FLASH)	30
A.5 Bandits/Active Learning	30
A.6 Dimensionality Reduction	30
A.7 RL/Transfer	30
A.8 Direct Search (Residual Check)	31
Appendix B: Surrogate Candidate Head-to-Head Arguments	31
B.1 Single CART vs Random Forest	31
B.2 EI vs 80/20 Heuristic	31
B.3 TPE Exclusion	31
B.4 QBC Exclusion	31
Appendix C: RL Landscape Analysis	32
Artifacts	32
References	32

0. Research Overview

I developed the FLASH-RF configuration performance tuning tool over seven iterations (v1–v7): four yielded substantial improvement, one yielded zero-degradation marginal improvement, and two were rejected by experiment. Each iteration follows “**identify problem** → **diagnose root cause** → **literature survey** → **algorithm design** → **experimental validation** → **result analysis** → **motivate next iteration**”. §2 provides the algorithm survey and selection derivation driving the first iteration’s design.

Final solution v6 is statistically significantly better than the random search baseline on all 8/8 systems, with a total hit-optimal count of **181/240**, systematically surpassing RS (35), HC (104), SA (67), BestConfig (53), and TPE (121) (see §10–§12 for details).

Iteration Roadmap:

Iter.	Version	Core Change	Key Finding
1	v1	RF(10) + mean ranking	7z failure (−2.1%), exposing 5 compounding root causes
2	v2	RF(20) + LCB + feature preprocessing	7z median turned positive (+1.68%, ns), 4/8 surpass TPE
3	v3	LightGBM ensemble + LCB	Rejected: uniformly worse than v2, proving bias reduction < variance reduction
4	v4	RF(20) + Thompson Sampling	3/8 significantly better than v2, LLVM 30/30 hit-optimal
5	v5	v4 + conditional log(1 + y)	Zero degradation, 7z median +3.7%

Iter.	Version	Core Change	Key Finding
6	v6	v5 + conditional KMeans stratified Phase-1	8/8 significantly > RS , total hits 181/240
7	v7	Conditional rank replacing conditional log	Rejected: 3 systems degraded, total hits -6

1. Problem Definition & Dataset

1.1 Problem Formalisation

Lab 3 defines the configuration performance tuning problem as (the impact of software configuration on system performance has been widely studied across multiple domains [1][5][6][18]):

$$x^* = \arg \min_{x \in X} f(x), \quad \text{s.t. } r \leq R$$

where $x = (x_1, x_2, \dots, x_n)$ is a configuration comprising n options within search space X (composed of binary, integer, and enumerated parameters), f is the performance function (runtime/latency), and R is the fixed measurement budget ($R = 100$ in this project). f is a black box: no gradients, no analytical form, and each query consumes 1 unit of budget.

1.2 Key Problem Characteristics

A few constraints shaped every algorithm decision I made. f is a black-box lookup table, so anything that needs gradients or an analytical form is out. The inputs are discrete (binary switches, integers, enumerations), ruling out methods that assume continuous spaces. And the budget is tight: with $R = 100$ measurements but up to 68,640 configurations, the largest system (7z) allows exploring only $\approx 0.15\%$ of its space. On the other end, brotli (180 configs) covers $\approx 56\%$. Any viable algorithm has to work across that full range. At least f is deterministic (same config, same result), so measurement noise is not a concern.

1.3 Dataset

8 real-world configurable software systems, from the CSV datasets provided by Lab 3:

System	Domain	$ X $	Features	Objective	Coverage $R/ X $
7z	Compression tool	68,640	8	Minimise runtime	0.15%
Apache	Web server	640	8	Minimise latency	15.6%
brotli	Compression tool	180	2	Minimise runtime	55.6%
LLVM	Compiler	65,536	16	Minimise runtime	0.15%
Postgres	Database	864	8	Minimise runtime	11.6%
spear	SAT solver	16,384	14	Minimise solve time	0.61%
storm	Stream processing	1,557	12	Minimise latency	6.4%

System	Domain	$ X $	Features	Objective	Coverage $R/ X $
x264	Video encoder	4,608	10	Minimise encoding time	2.2%

1.4 Dataset Processing

The first $n - 1$ columns of each CSV file are configuration parameters; the last column is the performance value. All systems are minimisation problems. The dataset serves as a “lookup table” simulating real measurements: given a configuration, query its performance value; valid configurations consume 1 unit of budget. I **sample directly from dataset rows**, ensuring every measurement is valid (no budget wasted on invalid configurations).

1.5 Experimental Evaluation Framework

Unified protocol across all iterations:

- **Budget:** $R = 100$ per run.
- **Repetitions:** Each algorithm runs **30 times** independently on each system, with deterministic seeds $s_i = 42i + 1$ ($i = 0, \dots, 29$).
- **Metric:** Best performance value found within budget (lower is better).
- **Statistical analysis: Median + IQR** over 30 runs; one-sided **Mann–Whitney U test** ($\alpha = 0.05$), $p < 0.05$ denoted *, $p < 0.01$ denoted **, $p < 0.001$ denoted ***.
- **Hit-optimal rate:** Number of runs (out of 30) that find the globally optimal configuration.
- **Improvement rate $\Delta\%$:** $\Delta\% = (f_{\text{RS}} - f_{\text{algo}})/f_{\text{RS}} \times 100\%$. Positive values indicate the algorithm outperforms RS (lower performance value); negative values indicate worse than RS. When the global optimum is 0 or close to 0 (e.g., spear), hitting the optimum can yield $\Delta\% = +100\%$, reflecting a qualitative leap from non-optimal to optimal rather than a literal performance doubling.

2. Algorithm Survey & Selection

2.1 Survey Scope

I wanted to make sure I was not overlooking some obvious algorithm family, so I worked through the major categories of optimisation methods covered in standard textbooks and review articles (Nocedal & Wright [24], Talbi [25], Shahriari et al. [26]), the software configuration literature (Gong & Chen 2024 [7]), and the broader ML-for-optimisation literature (Bengio et al. [27], Kotthoff [20]).

A large chunk of the optimisation literature turned out to be irrelevant up front. Methods that need an analytical form for f (mathematical programming, gradient-based methods) are out because f is a black-box lookup table. Methods that rely on geometric operations in continuous spaces (Nelder-Mead, pattern search, DIRECT) are out because the search space is discrete: you cannot reflect a simplex of binary vectors, and “line search” between two binary configurations is trivial. Multi-fidelity methods (Hyperband, BOHB

[14]) are out because the CSV lookup is all-or-nothing; there is no cheap approximation to evaluate. And full-dataset supervised learning trivially degenerates to sorting the CSV; it violates the $R = 100$ budget constraint that makes this problem interesting in the first place.

What survived: space-filling methods (RS, LHS, useful as baselines or initial phases), trajectory-based metaheuristics (HC, SA, which can work on discrete spaces but are limited in reach), population-based metaheuristics (GA, BestConfig, irace, though most classical populations need ≥ 200 evaluations), surrogate/model-based methods (GP-BO, SMAC, TPE, FLASH, the most promising family for sample efficiency), and selected ideas from bandits and active learning. A per-family analysis is in [Appendix A](#).

2.2 Surveyed Algorithm Families

I surveyed over 20 specific algorithms across the surviving families. The table below gives each family’s summary; full per-algorithm analysis is in [Appendix A](#).

Family	Representative Algorithms	Conclusion
Baseline	RS, LHS, Sobol, grid search	Initial phase only; pure coverage without learning, insufficient as standalone when $R \ll X $
Trajectory-based	HC, SA, tabu search, VNS	Locally viable, globally insufficient; limited reachable range under $R = 100$
Population-based	GA, DE, PSO, CMA-ES, BestConfig [3], irace [15]	Classical populations need too much budget (≥ 200 –500); BestConfig and irace viable but limited to region-level learning
Surrogate/model-based	GP-BO, SMAC [8], TPE [9], FLASH [2], COMBO [16], BOCS [17], neural surrogates [23]	Core family: predictive models enable configuration-level generalisation. CART fits discrete spaces natively; GP limited by stationarity assumption
Bandits/active learning	UCB, Thompson Sampling, QBC	Pure bandits cannot generalise across 68K arms; but bandit strategies can enhance surrogate methods
Dimensionality reduction	Morris screening, Sobol analysis, PCA+BO	Explicit screening too expensive ($\sim (n + 1) \times r$ budget); CART feature importance provides implicit reduction for free
RL/transfer	Q-Learning, DQN, cross-system transfer	RL requires thousands of episodes; transfer needs cross-system data that doesn’t exist
Direct search (residual check)	Nelder-Mead, Powell, MADS	Geometric operations degenerate to simpler methods on discrete spaces, confirming exclusion

2.3 Selection Rationale

After the survey I was left with a manageable set of candidates. Budget and structural constraints knock out most classical methods: GA needs ≥ 200 evaluations, CMA-ES requires continuous spaces, GP-BO’s kernel assumptions (stationarity, smoothness) are a poor match for software configurations with binary flags causing performance jumps. What remains is roughly: HC, SA, BestConfig, irace, TPE, RF-based surrogate methods (FLASH/SMAC family), and QBC.

Why surrogate methods over model-free ones. The key difference is in what the algorithm learns from its measurements. HC and SA use each measurement to decide a single local move; they do not build a predictive model of the performance landscape. BestConfig’s boundary narrowing and irace’s distribution narrowing are a step up, but they still learn “which regions are roughly good”, not “what will configuration x score.” Surrogate methods train a model $\hat{f}$ that can predict the performance of any unmeasured configuration, spreading the cost of every measurement across all future decisions.

For brotli ($R/|X| \approx 56\%$), model-free methods can stumble onto good configurations just by looking around. For 7z ($R/|X| \approx 0.15\%$), you need the model to generalise — there is no other way to cover 68K configurations with 100 measurements. Three properties of software performance landscapes give reason to expect a tree-based surrogate will work: a few dominant features tend to explain most of the variance (independently verified via ANOVA by Siegmund et al. [5]), the discrete variable types produce piecewise-constant performance patterns that CART is built to capture, and interactions are empirically low-order.

That said, this only holds if the surrogate is well-calibrated. When it is not, model guidance can be worse than random — the 7z failure in v1 (§3.6) is exactly this. But with reasonable model selection and regularisation, surrogate methods should have the edge.

I also considered hybrid strategies, e.g., running SA for the first 50 steps then switching to RF-BO, or running RF-BO and TPE in parallel with a budget split. The problem is that splitting the $R = 100$ budget hurts both components: you get a worse model (fewer training points) and fewer exploitation steps. The RF ensemble already provides internal hedging (10–20 trees averaging out individual errors), so a portfolio strategy buys little beyond what the ensemble already does. I opted for the simplest viable architecture.

Choosing among surrogate candidates: TPE vs FLASH-RF vs QBC. All three are validated sample-efficient methods, and the pure algorithm performance differences among them are limited. I went with FLASH-RF (Random Forest sequential optimisation, combining ideas from Nair et al. 2017 [2] and Hutter et al. 2011 [8]) for a mix of reasons:

- *Native discrete support.* RF’s CART base learners handle binary, integer, and categorical features directly. TPE does too (via categorical kernels), but its density-model internals are opaque. QBC needs an ensemble of CARTs, overlapping mechanistically with RF without adding clear benefit.
- *Ensemble regularisation.* With only $n \approx 30$ initial training samples and up to 68K configurations to predict, variance is the dominant error source. RF’s Bagging mechanism (Breiman 2001 [12]) directly attacks variance.
- *Implementation transparency.* For the coursework, I wanted the core loop visible: train the model, predict,

select, measure, repeat. Calling `optuna.TPESampler` hides the mechanism; implementing TPE from scratch (fitting two KDEs, handling categorical kernels, computing density ratios) is more complex than RF-based sequential optimisation.

- *Academic lineage.* FLASH [2] and SMAC [8] provide directly citable design rationale.

I should be honest: in an industry setting, using Optuna’s TPE would be a perfectly reasonable choice, arguably simpler. My preference for FLASH-RF is partly driven by the coursework requirement to demonstrate understanding, not by any basic weakness in TPE.

Two internal design decisions: (1) RF over single CART: a single CART with 30 training points overfits catastrophically on large spaces like 7z, while the RF ensemble averages out the variance; (2) the 80/20 exploit/explore heuristic over EI: EI is theoretically better but harder to implement correctly, and over only ~14 decision rounds the difference is hard to accumulate. I went with simplicity. Detailed arguments for both decisions and for the TPE/QBC exclusion are in [Appendix B](#).

2.4 Why CART over Gaussian Processes?

This is the most common question about the design, so I will address it directly. The surrogate is a Random Forest (10–20 CARTs); the argument targets the tree/CART piecewise surrogate.

Dimension	GP	CART/RF	Advantage
Non-stationarity	Kernel functions (RBF/Matérn) encode a global smoothness assumption, violated by binary flags causing performance jumps	Axis-aligned splits + piecewise-constant: naturally fits discrete jumps	CART
Discrete/categorical	Requires Hamming kernel or one-hot encoding; mixed types require kernel concatenation	Direct 0/1 splits, no preprocessing needed	CART
Computational cost	Training $O(n^3)$, prediction $O(n^2)/\text{point}$	Training $O(np \log n)$, prediction $O(\text{depth})$, millisecond-level	CART
Interpretability	Smooth posterior, no structured decomposition	Tree structure reveals feature importance and thresholds	CART

Improved GP variants (GP + Hamming kernel, COMBO [16], BOCS [17]) each mitigate some limitations

but none simultaneously handles mixed types, non-stationarity, and implementation simplicity. On purely binary spaces like LLVM, GP + Hamming kernel is a viable alternative; choosing RF there is more about cross-system consistency.

One genuine GP advantage: under deterministic f , GP can achieve exact interpolation ($\sigma_n^2 = 0$). But this benefit concentrates near observed points; the real challenge is predicting far from training data, where kernel smoothness assumptions create systematic errors at performance discontinuities. I would have liked to run a head-to-head GP vs RF experiment on LLVM, but did not get to it.

2.5 Limitations

My survey covers the major textbooks and the software configuration literature, but I do not claim it is exhaustive. The biggest gaps: I lack head-to-head experiments between FLASH-RF and TPE during the selection phase (though later iterations provide multiple rounds of TPE comparison; see §3.4, §4.5); I never ran a QBC controlled experiment; and the GP exact interpolation experiment is missing. Hyperparameter choices in v1 rely on defaults validated through single-factor sensitivity analysis (§3.7) but lack multi-factor joint optimisation.

3. Iteration 1: v1 — FLASH Framework + RF Ensemble

v1 is my first attempt at putting the §2 selection into practice: a straightforward two-phase RF sequential optimiser. It works on 7 of 8 systems, which was encouraging, but 7z turned out worse than random search (-2.1%). That one failure ended up driving everything that follows.

3.1 Starting Point & Motivation

Lab 3 requires finding the optimal configuration within a budget of $R = 100$. The baseline is Random Search, uniformly sampling 100 configurations and returning the best. For 7z (68,640 configs), 100 measurements cover only 0.15% of the space, making the probability of hitting the optimum extremely low. I need to **learn** from what I have already seen and focus the remaining budget on the most promising regions.

The algorithm selection in §2 leads directly to v1: adopt the FLASH [2] CART surrogate framework and incorporate the RF ensemble from SMAC [8].

3.2 Algorithm Design

v1 architecture: Two-phase sequential optimisation

Phase 1 (30% budget): Randomly sample 30 configurations $\rightarrow$ initial training set

Phase 2 (70% budget): RF-guided search

- └─ Train RF($n_{\text{estimators}}=10$)
- └─ Predict performance for all unmeasured configurations $\rightarrow$ take predicted mean
- └─ Select batch(5): 80% exploitation (predicted best) + 20% exploration (random from predicted-poor candidates)
- └─ Measure $\rightarrow$ update training set $\rightarrow$ repeat

Key design parameters:

Parameter	Value	Rationale
n_estimators	10	SMAC [8]’s RF ensemble approach (FLASH [2] uses a single CART)
max_depth	$\min(n_{features}, 10)$	Prevent overly deep overfitting with small samples
min_samples_leaf	$\max(1, n_{train}/10)$	Limit minimum samples per leaf
Exploit/explore ratio	80/20 fixed	FLASH [2]’s empirical default. In batch size 5, allocates 4 exploitation + 1 exploration
Initial exploration ratio	30%	RF needs sufficiently diverse training data. For LLVM ($n = 16$ binary features), the union bound on “at least 1 feature with no variation” at m samples is $16 \times 2^{1-m}$; setting this ≤ 0.05 yields $m \geq 10$. Taking practical split quality into account, I take 30 as a conservative choice. Going above 30% compresses the exploitation phase with diminishing return
Batch size	5	Amortises RF training cost; ~14 model updates during 70 measurements ¹

Dataset processing: Integer configurations from the CSV are passed directly to RF without any preprocessing.

3.3 Main Experimental Results**Table 1.** v1 main experimental results (30 runs, $R = 100$).

System	RS Median (IQR)	v1 Median (IQR)	$\Delta\%$	p -value	Significance
7z	4,576.3 (401.6)	4,673.8 (430.0)	−2.1	0.763	—
Apache	31.09 (0.33)	30.74 (0.07)	+1.1	9.8×10^{-5}	***
brotili	1.472 (0.054)	1.46 (0.00)	+0.8	4.1×10^{-7}	***
LLVM	59,141.5 (3,022.4)	52,285.4 (0.0)	+11.6	2.6×10^{-12}	***
Postgres	46,059.6 (96.1)	45,939.8 (108.0)	+0.26	1.1×10^{-4}	***
spear	0.00099 (0.000)	0.00099 (0.001)	0.0	5.2×10^{-4}	***
storm	0.000 (0.000)	0.000 (0.000)	0.0	3.7×10^{-4}	***
x264	22.759 (0.834)	21.586 (0.094)	+5.2	1.5×10^{-10}	***

¹I also tried batch=1 and batch=10; §3.7 has the sensitivity analysis. The difference turned out to be minimal.

Note: spear and storm have identical medians/IQRs on the surface yet highly significant p -values: the Mann-Whitney U test compares the full distribution rankings, not just medians. Both methods have many runs hitting the global optimum, making medians coincide, but the distribution tails differ significantly.

3.4 TPE Controlled Experiment

TPE uses Optuna TPESampler (`n_startup_trials = 30`, matching FLASH-RF’s 30% initial phase), all 8 systems.

System	RS Median	v1 Median	TPE Median	v1 vs RS	TPE vs RS	v1 vs TPE
7z	4,576.3	4,673.8	4,309.1	ns	***	ns
Apache	31.09	30.74	30.74	***	***	ns
LLVM	59,141.5	52,285.4	53,741.9	***	***	***
Postgres	46,059.6	45,939.8	45,939.8	***	***	ns
brothli	1.472	1.46	1.46	***	***	ns
spear	0.00099	0.00099	0.0	***	***	ns
storm	0.000	0.000	0.000	***	ns	*
x264	22.759	21.586	21.586	***	***	ns

Summary: v1 significantly better on 2/8 (LLVM***, storm*); TPE significantly better on 2/8 (7z, spear; reverse tests confirm); no significant difference on 4/8. The two trade wins, consistent with my §2.3 judgement that the performance gap between them is limited. v1 has an edge in purely binary high-dimensional spaces (LLVM), while TPE does better on 7z’s mixed-type large space.

3.5 Result Analysis

What works: v1 achieves statistically significant improvements on **7 out of 8** systems. LLVM is the star (+11.6%, 25/30 hit-optimal), as RF’s binary splits map almost perfectly onto 16 Boolean compiler passes.

What does not: 7z is the sole system where v1 **performs worse than RS** (−2.1%). Understanding why became the central question for v2.

I simultaneously implemented HC, SA, and BestConfig (excluded in §2.3 but retained as controls; implementations in `flash_tuner.py`). The full comparison with v6 is in [§10.1](#).

3.6 Root-Cause Diagnosis of the 7z Failure

I spent a while digging through data distributions, model behaviour, and code logic to understand what went wrong on 7z. The failure is not a single bug; it is several issues stacking on top of each other.

1. **Extremely skewed performance distribution.** 7z’s optimum is 4,196 but its median is ~35,106 and its max is 424,575. The optimal region is a tiny corner of the space. With 30 random initial samples, the chance of landing near it is low, and the training signal gets drowned by the mass of mediocre-to-bad configurations. LLVM has a comparable space size (65,536) but is purely binary, so CART can identify dominant pass effects with very few samples.
2. **Categorical variable encoding defect.** 7z’s `new_column` $\in \{1, 2, 3, 4, 5\}$ is an unordered categorical, but I was feeding it to RF as an integer. RF performs `new_column <= 2.5` style splits, implying a non-existent

ordinal relationship. If the true pattern is “categories 1,3,5 are good; 2,4 are bad”, threshold splitting needs two splits to separate them, while OHE needs one. Under limited `max_depth`, this extra depth costs capacity for capturing other feature interactions.

3. **Over-regularisation.** `min_samples_leaf = n_train // 10  $\approx$  6` forces rare “good” samples to share a leaf with many bad ones. A leaf containing 1 optimal configuration (4,200) and 5 configs at 12,000–35,000 predicts $\approx$ 17,367, burying the optimum.
4. **No uncertainty guidance in the acquisition strategy.** `v1` ranks candidates by predicted mean only, ignoring inter-tree variance. When predictions are wrong (due to the above issues), mean-greedy faithfully follows the erroneous ranking. The exploration pool is also biased, sampling from “predicted-poor” candidates rather than the full space, so model blind spots stay blind. If inter-tree variance were used (LCB/EI), high uncertainty in sparse regions would drive exploration and break the bias cycle.
5. **Non-equidistant BlockSize.** 13 levels spanning 1 to 4096 (approximately 2^k); on a linear scale, the first 7 levels occupy only 1.5% of the range. A $\log_2$ transform would equalise the splits.

These compound: the skewed distribution + encoding defect + over-regularisation produce bad predictions, then the mean-greedy acquisition locks onto the bad predictions, and the biased exploration pool prevents correction. LLVM is unaffected because purely binary features eliminate the encoding and scale issues. TPE is unaffected because it uses categorical kernels (no encoding defect) and density-ratio sampling (not ranking-driven).

3.7 *v1 Hyperparameter Sensitivity Analysis*

On 4 representative systems (brotli, x264, LLVM, 7z), using one-factor-at-a-time, 30 runs per group. Default: `init_ratio=0.30`, `batch_size=5`, `exploit_ratio=0.80`.

Initial exploration ratio:

init_ratio	brotli $\Delta\%$	x264 $\Delta\%$	LLVM $\Delta\%$	7z $\Delta\%$
0.10	+0.82%	+5.15%	+11.59%	−5.81%
0.15	+0.82%	+5.04%	+11.59%	−3.07%
0.20	+0.82%	+4.93%	+11.59%	−1.64%
0.25	+0.82%	+5.04%	+11.59%	−2.21%
0.30 ← default	+0.82%	+5.15%	+11.59%	−2.13%
0.40	+0.82%	+4.87%	+11.59%	−1.75%
0.50	+0.82%	+4.66%	+11.59%	−2.47%

Batch size:

batch_size	brotli $\Delta\%$	x264 $\Delta\%$	LLVM $\Delta\%$	7z $\Delta\%$
1	+0.82%	+4.90%	+11.59%	−2.47%
3	+0.82%	+4.59%	+11.59%	−1.07%

batch_size	brofli $\Delta\%$	x264 $\Delta\%$	LLVM $\Delta\%$	7z $\Delta\%$
5 ← default	+0.82%	+5.15%	+11.59%	−2.13%
10	+0.82%	+5.04%	+11.59%	−2.17%
20	+0.82%	+5.15%	+11.59%	−1.43%

Exploitation ratio:

exploit_ratio	brofli $\Delta\%$	x264 $\Delta\%$	LLVM $\Delta\%$	7z $\Delta\%$
0.50	+0.82%	+4.87%	+11.59%	−1.42%
0.70	+0.82%	+4.87%	+11.59%	−1.75%
0.80 ← default	+0.82%	+5.15%	+11.59%	−2.13%
1.00	+0.82%	+5.22%	+11.59%	−1.93%

brofli and LLVM are completely insensitive: brofli’s space is saturated, and LLVM’s binary structure lets RF model it precisely with few samples. x264 shows only slight degradation at init_ratio=0.50 (exploitation budget compressed); other parameter impacts are minimal. **7z is the only system showing substantial sensitivity**: init_ratio=0.10 is worst (−5.81%), validating the 30% choice; batch size impact is minimal (batch=1’s 70 RF training runs vs batch=5’s 14 bring no performance gain); exploit_ratio shows a trend **opposite** to x264: 0.50 (−1.42%) outperforms 0.80 (−2.13%), confirming that when the model is wrong, more exploration helps. Pure exploitation (1.00) makes 7z’s IQR explode. The default (0.30/5/0.80) sits near the optimum on 3/4 systems.

3.8 Remaining Issues & Next Steps

The §3.6 diagnosis points to four fixable targets for v2, prioritised by severity:

§3.6 Root Cause	Severity	v2 Fix Direction	Literature Source
Root Cause 2: Categorical encoding defect	Critical	OHE for categorical variables	SMAC [8]’s categorical handling
Root Cause 4a: No uncertainty guidance	Severe	LCB acquisition $\mu - \kappa\sigma$	Bayesian Optimisation [26]
Root Cause 3: Over-regularisation	Moderate	Relax min_samples_leaf + RF(20)	Breiman (2001) [12]
Root Cause 4b: Exploration pool bias	Moderate	Uniform random from full space	Exploration-exploitation theory
Root Cause 5: Non-equidistant BlockSize	Moderate	$\log_2$ transform	Standard feature engineering

Root Cause 1 (extremely skewed performance distribution) is an intrinsic data property and cannot be fixed at the feature level; it gets addressed later in v5.

4. Iteration 2: v2 — RF-LCB + Feature Preprocessing

The §3.6 diagnosis gave a clear fix list: OHE for categoricals, $\log_2$ for geometric features, LCB for uncertainty-guided acquisition, and full-space exploration. v2 applies all of them at once. 7z's median did flip from negative to positive (+1.68%); progress, but the p-value still would not cross the significance threshold. On the bright side, 4/8 systems now beat TPE.

4.1 What Changed

v2 targets the fixable root causes from §3.6: encoding defect and scale issue get merged into an automatic FeaturePreprocessor; over-regularisation is addressed with relaxed `min_samples_leaf` and a larger ensemble (20 trees).

4.2 Algorithm Design

Phase 1 (30% budget): Random sampling $\rightarrow$ initial training set (same as v1)

Phase 2 (70% budget): RF-LCB guided search

- └ FeaturePreprocessor: OHE(categorical) + $\log_2$ (geometric) + keep(binary/ordinal)
- └ Train RF(`n_estimators=20`, `oob_score=True`)
- └ Per-tree prediction $\rightarrow \mu(x), \sigma(x) = \text{tree_preds.std}()$
- └ Adaptive κ : quadratic decay based on OOB R^2 and search progress
 - | $\kappa = (1.5 - 1.2 \cdot \text{progress}) \times \max(0.1, (1 - \text{OOB})^2)$
- └ LCB = $\mu - \kappa \cdot \sigma$
- └ Select batch: exploit(top LCB) + explore(uniform random)
 - | `explore_ratio` = 0.4 if OOB < 0 else 0.2 (OOB safety net)
- └ Measure $\rightarrow$ update training set $\rightarrow$ repeat

Component	v1	v2	Why
Feature preprocessing	None	OHE + $\log_2$	Fix encoding defect, equalise scale
Ensemble size	10	20 trees	More stable σ estimation for LCB
Acquisition function	Predicted mean ranking	$\text{LCB} = \mu - \kappa \cdot \sigma$	Uncertainty guidance
<code>min_samples_leaf</code>	$n/10$	$\max(2, n/20)$	Relaxed, minimum 2 avoids single-sample σ noise
Exploration	From predicted-poor	Uniform random, full space	Remove bias

4.3 Dataset Processing Changes

Variable Type	Detection Rule	Encoding	Example
Binary	2 unique values	Keep as-is	LLVM’s passes $\{0, 1\}$
Geometric sequence	Adjacent ratio > 1.5 , approximately constant	$\log_2(x)$	7z’s BlockSize $\rightarrow$ $\{0, \dots, 12\}$
Categorical	3–6 unique values, consecutive integers	One-Hot Encoding	7z’s new_column $\rightarrow$ 5 binary columns
Other	None of above	Keep as-is	—

7z’s 8 original features expand to 13 dimensions after OHE.

4.4 Experimental Design & Execution

Two corrections during development: (1) the initial linear κ schedule caused over-exploration on systems where the model is already good (e.g., LLVM); switched to OOB R^2 -driven quadratic decay; (2) `min_samples_leaf = 1` produced extreme σ values from single-sample leaf nodes; raising the minimum to 2 fixed this. I probably should have caught the `min_samples_leaf=1` issue before running the full experiment — it wasted a day of compute.

4.5 Experimental Results

Table 2. v2 full comparison results (8 systems $\times$ 30 seeds, $R = 100$).

System	RS Median	v1 $\Delta\%$	v2 $\Delta\%$	TPE $\Delta\%$	v2 > RS	v2 > TPE	v2 Hit-Optimal
7z	4,576.3	−2.13%	+1.68%	+5.84%	ns	ns	0/30
Apache	31.09	+1.11%	+1.11%	+1.11%	***	*	30/30
LLVM	59,141.5	+11.59%	+11.59%	+9.13%	***	**	18/30
Postgres	46,059.6	+0.26%	+0.17%	+0.26%	***	ns	5/30
brotli	1.472	+0.82%	+0.82%	+0.82%	***	ns	29/30
spear	0.001	0.00%	+100%	+100%	**	ns	16/30
storm	0.000	0.00%	0.00%	0.00%	**	*	27/30
x264	22.76	+5.15%	+5.29%	+5.15%	***	*	22/30

4.6 Result Analysis

7z turned from failure to positive improvement (−2.13% $\rightarrow$ +1.68%). OHE eliminated the spurious ordinal relationship; $\log_2$ made BlockSize splits more uniform. **spear breakthrough** (0.00% $\rightarrow$ +100%): median dropped to the global optimum, 16/30 hit-optimal, as LCB’s uncertainty guidance made the search more effective. **x264 further improved** (+5.15% $\rightarrow$ +5.29%): hit-optimal jumped from 12/30 to 22/30, meaning the full-space uniform exploration reduced search bias in this medium-sized space. **v2 significantly outperforms TPE on 4/8 systems** (Apache, LLVM, storm, x264); TPE significantly better on 1/8 (7z, reverse $p \approx 0.003$).

PostgreSQL showed a slight decline in hit-optimal (8/30 $\rightarrow$ 5/30): LCB’s σ term introduced minor ranking noise on this system where the model was already accurate. Not statistically significant, but worth noting.

4.7 Remaining Issues

Despite the improvements, a few things bugged me:

#	Remaining Issue	Nature
1	OHE dimensionality expansion (7z: 8→13 features)	Engineering pain point
2	Inter-tree variance is not true probabilistic uncertainty: when all trees are equally ignorant about an unknown region, σ can be spuriously low	Theoretical limitation
3	RF (Bagging) only reduces variance, not bias: $MSE = Bias^2 + Var$; v2 primarily targets Var, Bias remains untouched	Theoretical opportunity
4	LCB's κ requires manual tuning; adaptive κ relies on OOB R^2 , but OOB itself is unstable at $n = 30$	Engineering fragility
5	LLVM hit rate slightly decreased (25/30 → 18/30): LCB's σ term introduces ranking noise	Minor regression

Issue 3 raised a natural question: if bias is the remaining error source, could Boosting help?

5. Iteration 3: v3 — LightGBM Attempt (Rejected)

I thought I could get more out of the surrogate model by switching from RF (Bagging) to LightGBM (Boosting), betting on bias reduction. v3.0 failed catastrophically; v3.1 recovered somewhat but still never beat v2 on any system. **Rejected.** Turns out, at $n \leq 100$, variance is the real enemy, not bias.

5.1 Motivation: Can Boosting Beat Bagging?

v2's RF uses Bagging: K CARTs trained independently then averaged. Bagging's mathematical property is variance reduction:

$$\text{Var}(\bar{f}) = \rho\sigma^2 + \frac{(1-\rho)\sigma^2}{K}$$

where ρ is inter-tree correlation. But Bagging does not reduce bias: the average of 20 shallow trees retains the modelling insufficiency of a single shallow tree. The MSE decomposes as:

$$MSE(\hat{f}) = \underbrace{Bias^2(\hat{f})}_{\text{v2 untouched}} + \underbrace{Var(\hat{f})}_{\text{v2 already reduced}}$$

The hypothesis: if bias is the dominant residual error, Boosting (sequentially fitting residuals) should yield further improvement.

5.2 Why LightGBM?

Before committing to LightGBM I checked whether anything outside the tree family could work at $n = 30$ –100. Linear models are too rigid for piecewise-constant structure. Neural networks have orders of magnitude too many parameters for 30 samples. GP has the stationarity issue from §2.4. k-NN at $n = 30, d = 16$ is hopeless. So: trees.

Among XGBoost, LightGBM, and CatBoost, I went with LightGBM [13] because it natively supports categorical features (eliminating the OHE dimension inflation that bothered me in v2), provides quantile regression for uncertainty estimation, and is well-maintained.²

The risk mitigation plan: keep each individual booster small ($\text{num_leaves} \leq 8$, few rounds), wrap the whole thing in a Bagging ensemble ($K=15$ independent LightGBM models), and use inter-model standard deviation as σ , structurally similar to v2’s RF.

5.3 v3.0: Quantile Regression — Complete Failure

The first attempt (v3.0) trained two independent LightGBM quantile models (one for the median, one for the 10th percentile) and used the gap as uncertainty. With DART mode and 80 boosting rounds, the results were terrible:

System	v2 Hit-Optimal	v3.0 Hit-Optimal	Trend
LLVM	18/30	4/30	Severe degradation
x264	22/30	3/30	Severe degradation
Apache	30/30	~14/30	Degradation

The problem: two independent quantile models each trained on 30 samples produce noisy uncertainty estimates, and 80 rounds of boosting on 30 samples means the later trees are purely fitting noise.

5.4 v3.1: Bagging of Boostings

I abandoned quantile regression and fell back to Bagging of Boostings: $K=15$ LightGBM models, each trained on a bootstrap subset, with μ = mean and σ = std of predictions, mechanistically isomorphic to v2’s RF. Boosting rounds adaptive to n : only 10 at $n = 30$ to limit overfitting.

Table 4. v3.1 results (30 runs, $R = 100$).

System	v2 $\Delta\%$	v3.1 $\Delta\%$	v3.1>v2	v3.1 opt
7z	+1.68%	−0.20%	ns	0/30
Apache	+1.11%	+0.87%	ns	14/30
LLVM	+11.59%	+11.59%	ns	16/30
Postgres	+0.17%	+0.13%	ns	4/30
brotli	+0.82%	+0.82%	ns	25/30

²CatBoost’s ordered boosting might have been a better choice in retrospect, but I did not have time to test it.

System	v2 $\Delta\%$	v3.1 $\Delta\%$	v3.1>v2	v3.1 opt
spear	+100%	+0.00%	ns	4/30
storm	+0.00%	+0.00%	ns	24/30
x264	+5.29%	+2.50%	ns	2/30

v3.1 vs v2: **0/8 significant improvements**. Much better than v3.0, but still fails to surpass v2 on any system.

5.5 Why Boosting Lost

At $n \leq 100$, variance dominates — a well-known consequence of the bias-variance decomposition in statistical learning [21]. Shallow trees with `num_leaves=8` already have limited bias; piecewise-constant models are a reasonable approximation for software performance landscapes. And Boosting’s sequential structure actually introduces extra variance: each tree fits the residuals of preceding trees, so errors propagate. RF’s trees train independently, so one tree’s noise stays contained.

In hindsight, the hypothesis was wrong: the bottleneck was not model quality.

5.6 Takeaway

v3’s failure narrowed the search: the model is good enough, and the real problem is in how I use it. If the surrogate is already near-optimal, the remaining leverage is in the acquisition strategy. Can I get rid of LCB’s fragile κ parameter entirely? That question launched v4.

6. Iteration 4: v4 — Thompson Sampling

v3’s failure convinced me the model itself is close to its ceiling, so v4 keeps RF and instead rethinks how I use the model’s predictions: Thompson Sampling replaces LCB. 3/8 systems significantly improve over v2, and LLVM hits 30/30.

6.1 Why Thompson Sampling?

LCB has one central problem: κ controls the exploration-exploitation tradeoff, and I have no principled way to set it. v2’s adaptive schedule was better than a constant, but it was still a heuristic built on OOB R^2 , which is itself noisy at $n = 30$.

Thompson Sampling (Thompson 1933 [10]) avoids this entirely. The idea: instead of averaging all 20 trees and then subtracting $\kappa\sigma$, just randomly pick a subset of trees and use their mean as the prediction. When trees agree, the subset mean is stable (exploitation). When they disagree, different subsets give different answers (exploration). No κ needed. The exploration-exploitation balance falls out of the posterior width automatically, and Russo & Van Roy (2014) [11] prove it achieves near-optimal Bayesian regret bounds.

This works because the problem is a stateless bandit, not an MDP — measuring one configuration does not change another’s performance, so there are no state transitions. RL methods like PPO or DQN need state transitions and thousands of episodes; I have 100 steps in a single run. The mismatch is total. ([Appendix C](#) has the full argument.)

6.2 Algorithm Design

Phase 1 (30% budget): Random sampling $\rightarrow$ initial training set (same as v1/v2)

Phase 2 (70% budget): RF + Thompson Sampling guided search

```

└─ FeaturePreprocessor: OHE + log2 (same as v2)
└─ Train RF(n_estimators=20)
└─ Thompson Sampling acquisition (batch version):
  |   for each slot in batch:
  |     1. Randomly draw K trees (K adaptive: early few  $\rightarrow$  late many)
  |     2. Mean prediction of K trees  $\rightarrow$  sampled_pred
  |     3. Select the unmeasured config with lowest sampled_pred
  |     4. Mark as selected (avoid intra-batch duplicates)
└─ Measure  $\rightarrow$  update training set  $\rightarrow$  repeat

```

Adaptive sub-sampling size:

$$K = \max(1, \lfloor 20 \times (0.2 + 0.6 \cdot \text{progress}) \rfloor)$$

Search Phase	progress	K	Behaviour
Early	0.0	4	Only 4/20 trees $\rightarrow$ strong exploration
Middle	0.5	10	Balanced
Late	1.0	16	16/20 trees $\rightarrow$ strong exploitation

No κ , no explore_ratio, no OOB safety net: all eliminated.

6.3 Experimental Results

Table 5. v4 complete results (30 runs, $R = 100$).

System	v2 Med	v4 Med	v2 $\Delta\%$	v4 $\Delta\%$	v4>RS	v4>v2	v4 opt
7z	4499.2	4536.0	+1.68%	+0.88%	ns	ns	2/30
Apache	30.74	30.74	+1.11%	+1.11%	***	—	30/30
LLVM	52285.4	52285.4	+11.59%	+11.59%	***	***	30/30
Postgres	45980.8	45939.8	+0.17%	+0.26%	***	*	9/30
brotili	1.460	1.460	+0.82%	+0.82%	***	ns	30/30
spear	0.000	0.000	+100%	+100%	***	**	25/30
storm	0.000	0.000	+0.00%	+0.00%	***	ns	29/30
x264	21.556	21.556	+5.29%	+5.29%	***	ns	20/30

6.4 Result Analysis

3/8 systems significantly outperform v2, the first significant improvement in acquisition strategy across four iterations:

System	v2 Hit-Optimal	v4 Hit-Optimal	<i>p</i> -value
LLVM	18/30	30/30	7.3×10^{-5} ***
spear	16/30	25/30	0.006 **
PostgreSQL	5/30	9/30	0.014 *

Why LLVM benefits most: LLVM’s purely binary features produce strong inter-tree agreement and universally small σ . LCB with small σ degenerates to pure greedy exploitation. TS breaks this by randomly sub-sampling trees, re-introducing exploration even when σ is low.

7z declined slightly (+1.68% $\rightarrow$ +0.88%, ns) and x264 dropped from 22/30 to 20/30 (ns); TS’s early strong exploration ($K = 4$ trees) adds randomness that imposes a minor cost on systems where the model was already accurate enough.

6.5 v3 vs v4 — Model vs Strategy

Dimension	v3 (changed model)	v4 (changed strategy)
What changed	RF $\rightarrow$ LightGBM ensemble	LCB $\rightarrow$ Thompson Sampling
What stayed	Same LCB strategy as v2	Same RF model as v2
vs v2	0/8 significant improvements	3/8 significant improvements
Lesson	Model side is near-optimal	Strategy side still has room

7. Iteration 5: v5 — Conditional Log Transform

I tried three candidate fixes for v4’s remaining issues; two of them made things worse. Only conditional $\log(1 + y)$ survived, giving 7z a modest +3.7% median boost without hurting anything else. But the more interesting outcome was one I did not plan for: the diagnosis pointed me toward a dimension I had been completely ignoring: the structure of the unlabelled X space.

7.1 Gap Analysis

System	v4 opt	Saturation	Judgement
Apache	30/30	100%	Saturated
brotili	30/30	100%	Saturated
LLVM	30/30	100%	Saturated
storm	29/30	~97%	Near-saturated
spear	25/30	83%	Room to improve
x264	20/30	67%	Room to improve
Postgres	9/30	30%	Significant room
7z	2/30	7%	Core challenge

7.2 Diagnosis

- **Root Cause A: Target skew hurts RF splits (primarily 7z).** 7z’s performance distribution is extremely skewed: $f \in [4196, 424575]$, median ≈ 35106 , skewness ≈ 2.68 . RF’s MSE split criterion gets dominated by extreme values, prioritising “very bad vs mediocre” splits rather than “mediocre vs good.” All of v2–v4 failed to address the output-side distribution, a gap symmetric to input-side preprocessing.
- **Root Cause B: Fixed batch size wastes information efficiency (primarily PostgreSQL).** Fixed batch=5 means RF makes the latter 4 decisions using a model that is “5 steps stale.” PostgreSQL’s coverage rate is 11.6%, meaning each measurement carries substantial marginal information; the cost of 5-step information lag is higher than on low-coverage systems.
- **Root Cause C: TS global sampling incurs tail exploration cost on small spaces (affects x264).** x264 has only 4,608 configurations. TS in early stages uses $K = 4$ trees (20% sub-sampling), with high randomness; on a “medium-to-small” space this may over-explore unnecessary regions. v2’s LCB happened to fit x264 (when OOB R^2 is moderate, κ decreases, degenerating to mean ranking); TS’s extra randomness in this scenario is slightly “over-exploratory.”
- **Root Cause D: Phase-1 uniform random sampling has poor coverage (large spaces).** On 7z, 30 random points do not guarantee coverage of all feature-value combinations; some feature regions may be entirely uncovered, forcing RF to extrapolate.

7.3 What I Tried

The diagnosis pointed to 8 candidate directions. I shortlisted the top 3 for implementation:

#	Direction	Target Root Cause	Target Systems	Expected Benefit	Risk
1	Target value transform $\log(1 + y)$	A	7z	Medium-High	Low (monotone, preserves ordering)
2	Adaptive batch size (1→3→5)	B	Postgres	Medium	Medium (may damage TS diversity)
3	Local search refinement (Hamming-1)	C	x264	Medium	Medium (too early → local optimum trap)
4	Stratified initial sampling (KMeans)	D	7z, large spaces	Medium	Low (worst case = random)
5	Intra-batch diversity constraint	—	Global	Low	Medium

#	Direction	Target Root Cause	Target Systems	Expected Benefit	Risk
6	K sub-sampling schedule optimisation	—	Global	Low	Medium (new hyper- parameters)
7	Increase RF tree count (20→50)	—	Global	Low	Medium (smaller variance → less TS exploration)
8	Cross-system transfer learning	—	Global	0	Infeasible (no shared feature space)

I implemented the first 3; Direction 4 turned out to be the remaining blind spot (§7.5, validated in v6). Directions 5–8 were not pursued.

Ablation results:

- **All three enabled (adaptive batch 1→3→5 + $\log(1+y)$ + Hamming-1 local search):** spear dropped from 25/30 to 20/30, storm from 29/30 to 25/30. The problem: batch=1 kills TS’s diversity: with only 1 candidate per round, posterior sampling degenerates to greedy.
- **$\log(1+y)$ + batch=5 (removed adaptive batch):** spear improved to 27/30, storm to 30/30, but x264 dropped from 20/30 to 17/30. x264’s performance range is only $8.1\times$; log compresses the gap in the near-optimal region from 0.044 to 0.002, killing RF’s ability to discriminate there.

In hindsight, I should have tested each component individually first instead of combining all three. That would have saved a round of experiments.

- **Final scheme: conditional log transform (threshold $> 10\times$).** Compute the dynamic range at the end of Phase-1. Apply $\log(1 + y)$ only when it exceeds $10\times$:

System	Dynamic Range	Log Enabled	Reason
7z	$\sim 100\times$	Yes	Extreme skew
spear	∞ (contains zeros)	Yes	$\log(1+0)=0$ is fine
storm	∞ (contains zeros)	Yes	Same
x264	$8.1\times$	No	Would lose near-optimal resolution
Postgres	$2.1\times$	No	Narrow range
Apache/LLVM/brotli	—	No	Already saturated

7.4 Results

Table 6. v5 final scheme (8 systems $\times$ 30 seeds, $R = 100$).

System	v4 Med	v5 Med	v4 opt	v5 opt	v5>v4
7z	4,536.0	4,368.1	2/30	2/30	ns
Apache	30.74	30.74	30/30	30/30	ns
LLVM	52,285.4	52,285.4	30/30	30/30	ns
Postgres	45,939.8	45,939.8	9/30	9/30	ns
brotli	1.460	1.460	30/30	30/30	ns
spear	0.000	0.000	25/30	26/30	ns
storm	0.000	0.000	29/30	30/30	ns
x264	21.556	21.556	20/30	20/30	ns

Zero degradation, marginal improvement. The main lesson from the ablations: TS is sensitive to batch size (batch=5 is the minimum for effective diversity), and log transform is a double-edged sword (good for $100\times$ range, harmful for $8\times$).

7.5 Remaining Blind Spot

Working through the ablations, I realised that the marginal return from squeezing more out of measured data is diminishing. But reviewing the code, something stood out: **all versions' RF training uses only the measured (X, y) pairs, while tens of thousands of freely available unlabelled configurations are completely ignored during initial sampling.** The cluster structure and density distribution of the X space could inform a better Phase-1 sampling strategy. This became v6.

8. Iteration 6: v6 — Conditional Stratified Initial Sampling

Something had been bothering me since v5: tens of thousands of unlabelled configurations were sitting right there, and I was ignoring all of them during Phase-1. v6 uses KMeans clustering to guide initial sampling on large-space systems. The result justified the detour: for the first time across six iterations, **all 8/8 systems are statistically significantly better than RS**, with 181/240 total hits.

8.1 Motivation

For 7z (68,640 configs), 30 purely random points cover only 0.04% and may concentrate in certain regions while missing others. Using the structure of the full X space (which is free, no budget cost) to guide Phase-1 sampling should improve coverage.

8.2 Algorithm Design

Only Phase-1's sampling strategy changes; Phase-2 is untouched. Stratified sampling is enabled only when $|X| > 5000$ (coverage $< 2\%$).

1. Run `MiniBatchKMeans(n_clusters=min(15, init_size))`³ on the encoded full X space
2. Allocate quota per cluster proportionally: `n_pick = max(1, round(init_size * len(cluster) / len(X)))`

³Full KMeans was noticeably slower on 68K rows; MiniBatchKMeans converged in under a second.

3. Draw n_{pick} random configs from each cluster
4. Fill remainder (if any) with random samples from the full space

Affected systems: 7z ($|X| = 68640$), LLVM ($|X| = 65536$), and spear ($|X| = 16384$). The other 5 systems are unaffected.

8.3 Rejected Alternatives

I also tried pseudo-label training (generating KNN pseudo-labels for unlabelled X) and global stratification (enabling it on all systems). Both degraded storm and PostgreSQL: pseudo-labels injected noise into RF, and global stratification changed the RNG sequence on small-space systems, introducing fluctuation.

8.4 Experimental Results

Table 7. v6 results (8 systems $\times$ 30 seeds, $R = 100$).

System	$ X $	Stratified	v5 $\Delta\%$	v6 $\Delta\%$	v6>RS	v6>v5	v5 opt	v6 opt
7z	68,640	Yes	+4.55%	+5.92%	**	*	2/30	5/30
Apache	640	No	+1.11%	+1.11%	***	ns	30/30	30/30
LLVM	65,536	Yes	+11.59%	+11.59%	***	ns	30/30	30/30
Postgres	864	No	+0.26%	+0.26%	***	ns	9/30	9/30
brothli	180	No	+0.82%	+0.82%	***	ns	30/30	30/30
spear	16,384	Yes	+100%	+100%	***	ns	26/30	27/30
storm	1,557	No	+0.00%	+0.00%	***	ns	30/30	30/30
x264	4,608	No	+5.29%	+5.29%	***	ns	20/30	20/30

v6 significantly > RS: **8/8**. v6 significantly > v5: 1/8 (7z, $p = 0.040$). Zero degradation. Total hit-optimal: $177 \rightarrow \mathbf{181/240}$.

8.5 Result Analysis

7z’s IQR dropped from 4736.5 to 420.5, variance reduced by $11\times$. Stratified sampling ensures Phase-1’s 30 points cover all major regions of the X space. This is the first time in six iterations that all 8 systems are statistically significantly better than the baseline. The 5 unaffected systems have identical results to v5 across all 30 runs.

9. Iteration 7: v7 — Rank Transform (Rejected)

I wondered whether rank transform could eliminate distributional skew more thoroughly than log. It cannot: 3 systems degrade, total hits drop by 6. **Rejected.** In a way this failure was still useful: it confirmed that $\log(1 + y)$ had been doing more right than I initially appreciated, and v6 is where this project ends.

9.1 Motivation

v6’s conditional $\log(1 + y)$ still retains absolute value information. A more aggressive idea: replace it with $\text{rank}(y)/n$, which completely eliminates distributional skew and forces RF to learn purely the ordering. The change is a one-liner in the conditional branch: `np.log1p(y_raw)  $\rightarrow$  rankdata(y_raw) / len(y_raw)`.

9.2 Results

Table 8. v7 controlled experiment (8 systems $\times$ 30 seeds, $R = 100$).

System	v6 Hit-Optimal	v7 Hit-Optimal	Change
7z	5/30	2/30	−3 degraded
Apache	30/30	30/30	Unchanged
LLVM	30/30	30/30	Unchanged
Postgres	9/30	9/30	Unchanged
brotli	30/30	30/30	Unchanged
spear	27/30	26/30	−1 degraded
storm	30/30	28/30	−2 degraded
x264	20/30	20/30	Unchanged
Total	181/240	175/240	−6 degraded

3 systems degraded, 0 improved. 7z’s median worsened to below the RS baseline. **Rejected.**

9.3 Why It Failed

Two problems. First, **target instability**: $\log(1 + y)$ is a fixed pointwise transform: 4196 always maps to 8.34. But rank transform changes every time a new point is added in Phase-2, because all existing ranks reshuffle. RF faces drifting training targets. Second, **loss of distance information**: $\log(1 + y)$ preserves the sense that 4196 is much better than 9000 (8.34 vs 9.10). Rank maps 4196 to rank=1, 4200 to rank=2, 9000 to rank=15; RF cannot tell “close to optimum” from “far from optimum.”

The takeaway is simpler than it looks: the target transform has to be pointwise. Adding a new observation should not retroactively change existing targets. $\log(1 + y)$ and $\sqrt{y}$ satisfy this; rank transforms and quantile normalisation do not.

At this point, five optimisation dimensions have all been validated:

Dimension	Attempted Version	Conclusion	Depth
Surrogate model	v3 (LightGBM)	RF is optimal at $n \leq 100$	Full iteration
Acquisition strategy	v4 (Thompson Sampling)	TS eliminates LCB’s κ	Full iteration
Labelled data transform	v5 (log) / v7 (rank)	$\log(1+y)$ is the optimal balance	Two rounds
Unlabelled data utilisation	v6 (stratified sampling)	X space structure guides Phase-1	Full iteration
Budget allocation	v5 (adaptive batch)	batch=5 is TS’s minimum diversity unit	Single ablation

v6 is confirmed as the final solution.

10. Final Validation: Multi-Method Multi-Budget Comparison

§3–§9 focused on internal comparison, each iteration relative to the previous. This section validates v6 against mainstream baselines via external comparison.

10.1 6-Method Full Comparison ($R=100$)

6 methods: RS (random search baseline), HC (steepest-ascent hill climbing with restarts), SA (simulated annealing), BestConfig (DDS+RBS [3]), TPE [9] (via Optuna), FLASH-RF v6 (my method).

Table 9. Hit-optimal counts (30 runs, $R = 100$).

System	RS	HC	SA	BestConfig	TPE	v6	v6 Significantly Better Than
7z	0	0	0	1	2	5	RS** HC*** SA*** BC***
Apache	5	10	6	3	25	30	RS*** HC*** SA*** BC*** TPE*
brotli	12	30	21	19	30	30	RS*** SA*** BC***
LLVM	0	5	5	13	6	30	RS*** HC*** SA*** BC*** TPE***
Postgres	1	6	4	1	8	9	RS*** HC* SA*** BC***
spear	0	7	7	3	19	27	RS*** HC*** SA*** BC*** TPE**
storm	17	24	18	7	22	30	RS*** HC** SA*** BC*** TPE**
x264	0	22	6	6	9	20	RS*** SA*** BC*** TPE**
Total	35	104	67	53	121	181	—

v6 is significantly better than RS and SA on 8/8 systems, better than BestConfig on 8/8, better than HC on 6/8, and better than TPE on 5/8.

A few observations. HC does surprisingly well on brotli (30/30) and x264 (22/30), small spaces where neighbourhood search can cover a lot of ground. But it collapses on large spaces (7z, LLVM, spear). BestConfig actually does worse than RS on Apache (3/30) and storm (7/30) — I did not expect that from a published method with decent reported results, so seeing it lose to pure random was striking. Its divide-and-conquer sampling apparently struggles in discrete spaces. TPE (121/240) is the strongest non-RF method, and on 7z specifically TPE (2/30) and v6 (5/30) are not far apart ($p=0.58$, ns).

The overall pattern supports the §2.3 prediction: surrogate methods (v6: 181, TPE: 121) dominate model-free methods (HC: 104, SA: 67, BestConfig: 53, RS: 35).

10.2 Multi-Budget Scenarios ($R=30/100/200$)

Table 10. v6 hit-optimal under different budgets (30 runs).

System	R=30	R=100	R=200
7z	0	5	6
Apache	8	30	30
brotili	26	30	30
LLVM	29	30	30
Postgres	2	9	20
spear	10	27	28
storm	11	30	30
x264	4	20	18
Total	90	181	192

The $R=30 \rightarrow R=100$ jump is the largest (+91). $R=100 \rightarrow R=200$ shows diminishing returns (+11); 4/8 systems are already saturated at $R=100$. PostgreSQL benefits most from extra budget ($9 \rightarrow 20$).

x264 shows an anomalous decline at $R=200$ ($20 \rightarrow 18$). Extended data shows HC ($27/30$) and SA ($27/30$) both surpass v6 at $R=200$ on x264. This makes sense: when coverage gets high enough ($R/|X| \approx 4.3\%$), model-free methods’ local search can catch up with surrogate methods. The surrogate advantage concentrates in the low-coverage regime.

11. Global Review

11.1 Version Comparison

Iterations $v1-v4$:

Component	v1	v2	v3	v4
Surrogate	RF(10)	RF(20)	LGB(15)	RF(20)
Preprocessing	None	OHE+log ₂	Nat. cat+log ₂	OHE+log ₂
Acquisition	Mean rank	LCB (κ)	LCB (κ)	TS
Target transform	None	None	None	None
Phase-1 sampling	Random	Random	Random	Random
Sig. > RS	7/8	7/8	6/8	7/8
Total hits	—	—	—	—

Iterations $v5-v7$:

Component	v5	v6	v7 (rej.)
Surrogate	RF(20)	RF(20)	RF(20)
Preprocessing	OHE+log ₂	OHE+log ₂	OHE+log ₂
Acquisition	TS	TS	TS
Target transform	Cond. log	Cond. log	Cond. rank

Component	v5	v6	v7 (rej.)
Phase-1 sampling	Random	Cond. stratified	Cond. stratified
Sig. > RS	7/8	8/8	7/8 ↓
Total hits	177/240	181/240	175/240 ↓

11.2 Iteration Logic

v1: Can a model guide the search? → Yes (7/8 beat RS), but 7z fails

↓ Found: encoding defect + no uncertainty guidance

v2: Fix known defects → broad improvement, but model bias untouched

↓ Hypothesis: Can Boosting reduce bias?

v3: Try Boosting → fails → proves variance > bias at $n \leq 100$

↓ Insight: bottleneck is in strategy, not model

v4: Replace acquisition strategy → success → 3/8 significant improvement, LLVM 30/30

↓ Diagnosis: 4/8 saturated, remaining issues in data representation

v5: Conditional log transform → zero degradation, marginal improvement

↓ Insight: unlabelled X space completely unused

v6: Exploit X structure for stratified Phase-1 → 8/8 all significantly > RS

↓ Hypothesis: Can rank do better than log?

v7: Rank transform → fails → 3 systems degraded → confirms v6 as final solution

12. Conclusion

Looking back, the seven iterations split roughly into three kinds: the ones that worked (v1, v2, v4, v6), the one that barely moved the needle (v5), and the two that failed and got thrown out (v3, v7). The failures were arguably more instructive — v3 settled the variance-vs-bias question, and v7 taught me why rank transforms break sequential optimisation.

The final solution has four independently validated components:

Component	Source Iteration	Core Mechanism
RF surrogate model	v1→v2	Bagging ensemble (20 CARTs) + feature preprocessing (OHE + $\log_2$)
Thompson Sampling acquisition	v4	Random tree sub-sampling eliminates LCB's κ , achieving adaptive exploration-exploitation balance
Conditional $\log(1 + y)$ transform	v5	Compresses skew when dynamic range > $10\times$, preserving distance information

Component	Source Iteration	Core Mechanism
Conditional stratified initial sampling	v6	KMeans cluster structure of unlabelled X space guides Phase-1 sampling (large-space systems only)

A few results surprised me. The biggest one: at these sample sizes, variance matters far more than bias. I went into v3 expecting Boosting to outperform Bagging, and it lost across the board — the model was already about as good as it could get with 30–100 training points (§5.5). What actually moved the needle was changing *how* I used the model’s output. Replacing LCB with Thompson Sampling eliminated κ tuning entirely and pushed LLVM from 18/30 to 30/30 (§6.4).

The other surprise was how much free information I had been leaving on the table. All versions up to v5 threw away the structure of the unlabelled X space during initial sampling. Once I clustered it and sampled proportionally, 7z’s variance dropped by $11\times$ (§8.5). And v7’s failure with rank transforms confirmed something I should have seen earlier: the target transform needs to be pointwise — adding a new observation cannot retroactively change every existing training label (§9.3).

Final performance: v6 is statistically significantly better than RS on all 8/8 systems, with 181/240 total hit-optimal, consistently outperforming all 5 comparison methods.

Remaining limitations. Batch TS currently selects candidates independently within each batch; ideally the posterior should update after each selection. The 59/240 misses (7z 25/30 + PostgreSQL 21/30 + x264 10/30 + spear 3/30) are primarily constrained by $R = 100$ budget coverage (7z’s coverage is only 0.15%), not surmountable by algorithmic strategy. And I never ran the GP vs RF head-to-head experiment on LLVM that would have settled the §2.4 question empirically.

Appendix A: Per-Family Algorithm Survey Details

A.1 Baseline Methods (RS, LHS, Sobol, grid search)

Pure coverage methods with no learning. Grid search is infeasible ($3^8 = 6561 > R$ for $n = 8$). LHS and Sobol improve space-filling over RS but still spend all budget on sampling with no exploitation. Only suitable as Phase-1 of a two-stage approach.

A.2 Trajectory-Based (HC, SA, Tabu, VNS)

SA was actually the first thing I coded; I could get something running in 20 minutes. Watching it get stuck in mediocre regions on 7z was what really convinced me to move to surrogate methods. HC evaluates neighbours and moves to the best one; SA adds random acceptance to escape local optima but with only 100 steps, aggressive cooling makes it near-HC. Tabu search avoids revisiting recent solutions but still does not build a predictive model. Under $R = 100$, the search scope is confined to the reachable neighbourhood of the starting point.

A.3 Population-Based (GA, DE, PSO, CMA-ES, BestConfig, irace)

Classical population methods (GA, DE, PSO, CMA-ES) need large budgets: GA’s minimum viable population is 15–20 individuals (Goldberg [19]), and $R = 100$ allows only 5 generations. DE and PSO are designed for continuous spaces; CMA-ES’s covariance matrix is undefined on binary vectors.

BestConfig [3] and irace [15] are the viable exceptions. BestConfig’s DDS provides space coverage; irace’s statistical racing eliminates inferior configs. Both learn “which regions are roughly good” (region-level), but neither builds a configuration-level predictive model $\hat{f}(x)$. This distinction matters when coverage is low: at 0.15% coverage, you need the model to predict performance for the 99.85% you have not measured.

A.4 Surrogate/Model-Based (GP-BO, SMAC, TPE, FLASH)

The most promising family. GP-BO (Spearmin [4]) has the best-grounded uncertainty (posterior variance), but its kernel assumptions (stationarity, smoothness) are a poor fit for software configurations. SMAC [8] uses RF surrogates and natively handles discrete variables. TPE [9] uses density estimation and is well-packaged in Optuna. FLASH [2] uses a single CART with an 80/20 heuristic. The broader surrogate-assisted optimisation literature [22] confirms that tree-based surrogates are competitive when the input space is discrete and the evaluation budget is small.

I spent more time on this family than all others combined; it was clear early on that the real decision lay within this family, not between families.

A.5 Bandits/Active Learning

Pure bandits (UCB, TS) treat each configuration as an independent arm; with 68K arms and 100 pulls, most arms never get tried and there is no generalisation. But paired with a surrogate model, bandit strategies become powerful acquisition functions. In fact, that is exactly what v4 ends up doing: TS as a selection strategy on top of RF predictions. In the initial survey I dismissed TS too quickly (“same limitation, no cross-configuration generalisation”), not realising the RF would supply the generalisation.

A.6 Dimensionality Reduction

Morris screening costs $(n+1) \times r$ budget: for LLVM ($n = 16$) at $r = 5$ repetitions, that is 85 out of 100 budget gone before any optimisation starts. Sobol analysis needs even more. CART feature importance provides the same information as a free by-product of the surrogate.

A.7 RL/Transfer

RL needs thousands of episodes; I have 1 episode with 100 steps. Recent neural combinatorial optimisation methods (Pointer Networks [28], attention-based routing [29]) show promise on structured problems like TSP, but they require large offline training datasets of solved instances and assume a fixed combinatorial structure — neither of which applies here. Transfer learning needs cross-system data that does not exist. Not applicable.

A.8 Direct Search (Residual Check)

Nelder-Mead was actually the first method I looked up when starting this project; it is the go-to example in every undergraduate optimisation course. Working through why its simplex operations break on binary spaces (you cannot compute a centroid of binary vectors; $[0.5, 0.3, \dots]$ is not a valid configuration) was a useful exercise. It forced me to take the discrete constraint seriously. All direct search methods either become undefined on discrete spaces or degenerate to simpler methods already covered by HC/SA.

Appendix B: Surrogate Candidate Head-to-Head Arguments

B.1 Single CART vs Random Forest

A single CART trained on all 30 initial samples may be individually stronger than any bootstrap-subset tree in RF, so at first glance RF’s advantage is not obvious. The problem shows up on large spaces: on 7z (68,640 configs), a single CART with 30 samples must extrapolate to 68,610 unseen configurations. Small random fluctuations in split paths get amplified by the enormous extrapolation space; in experiments, single CART’s median is worse than the RS baseline by $\sim 85\%$. RF’s ensemble averaging suppresses this. The theoretical prediction (bootstrap sub-sampling might hurt at $n = 30$) was directionally correct for small spaces (brotli) but badly underestimated how much $|X|$ amplifies single-model variance.

B.2 EI vs 80/20 Heuristic

EI (Expected Improvement) is theoretically superior; it balances mean and variance in a principled way. I did sketch out an EI implementation early on; the Monte Carlo integral itself was straightforward, but handling edge cases (what happens when $\sigma(x) \approx 0$ across the board, as on LLVM where all 20 trees agree?) ate more debugging time than the rest of the acquisition pipeline. Over only 14 decision rounds the refinement EI offers is unlikely to accumulate into a measurable edge, so I dropped it.

B.3 TPE Exclusion

At the algorithm performance level, TPE and FLASH-RF trade wins. I excluded TPE primarily because: (1) calling Optuna hides the algorithmic mechanism, and for coursework I wanted the core loop visible; (2) TPE’s density ratios do not provide feature importance, while RF’s `feature_importances_` is useful for understanding which options matter. In an industry setting without the “demonstrate understanding” requirement, TPE via Optuna would be a perfectly reasonable choice.

B.4 QBC Exclusion

QBC’s “maximum disagreement” criterion maximises model uncertainty reduction, a model improvement objective, not directly an optimisation objective. Under $R_2 = 70$ sequential budget, if too many queries target high-disagreement but low-performance regions, exploitation suffers. The 80/20 heuristic devotes 56/70 steps to direct exploitation. Meanwhile, QBC needs a bootstrap ensemble (≥ 3 CARTs), mechanistically overlapping with RF but adding complexity without clear benefit. I should note that I never ran a QBC controlled experiment, so this is a qualitative exclusion.

Appendix C: RL Landscape Analysis

My problem has no state transitions: measuring configuration A does not change configuration B’s performance. There is no environment state that evolves over time, no delayed rewards (I get $f(x)$ immediately), and the “environment” is deterministic. This is not an MDP; it is a stateless bandit.

That rules out everything that needs state transitions: value-based methods (Q-Learning, DQN, which need $Q(s, a)$ over states), policy gradient (PPO, TRPO, which need advantage functions over states), actor-critic (A2C, SAC, which need both), model-based RL (Dyna-Q, MuZero, which model state transitions that do not exist), and so on. It also rules out meta-RL (needs a distribution of similar tasks), hierarchical RL (no sub-goal structure), multi-agent RL (single agent), and inverse RL (no expert demonstrations). Even within the bandit family, ϵ -greedy needs ϵ , UCB/LCB needs κ , and Boltzmann needs temperature. Thompson Sampling is the only one that needs no hyperparameters and naturally integrates with RF’s tree ensemble as posterior samples.

To put it bluntly: PPO is a policy gradient algorithm designed for MDPs with state transitions, requiring millions of steps to train a neural network policy. My problem has no state transitions, only 100 steps, and already has an RF surrogate. The two just do not line up at all.

Artifacts

Public repository (source code + raw experimental data): https://github.com/yelaiyuluo/ISE_Coursework_TairuiZhang

File	Content
iteration1/flash_tuner.py	v1 implementation + RS/HC/SA/BestConfig baselines
iteration1/run_ablation.py	TPE comparison + hyperparameter sensitivity
iteration2/flash_tuner_v2.py	v2 (RF-LCB + feature preprocessing)
iteration3/flash_tuner_v3.py	v3 (LightGBM, rejected)
iteration4/flash_tuner_v4.py	v4 (RF + Thompson Sampling)
iteration5/flash_tuner_v5.py	v5 (conditional log transform)
iteration6/flash_tuner_v6.py	v6 (final solution)
iteration6/run_extended_experiments.py	Multi-baseline multi-budget experiments
iteration7/flash_tuner_v7.py	v7 (rank transform, rejected)

References

- [1] P. Jamshidi, N. Siegmund, M. Velez, C. Kästner, A. Patel, and Y. Agarwal, “Transfer learning for performance modeling of configurable systems: An exploratory analysis,” in *Proc. ASE*, ACM, 2017, pp. 497–508.
- [2] V. Nair, T. Menzies, N. Siegmund, and S. Apel, “Using bad learners to find good configurations,” in *Proc. ESEC/FSE*, ACM, 2017, pp. 257–267.

- [3] Y. Zhu et al., “BestConfig: Tapping the performance potential of systems via automatic configuration tuning,” in *Proc. SoCC*, 2017, pp. 338–350.
- [4] J. Snoek, H. Larochelle, and R. P. Adams, “Practical Bayesian optimization of machine learning algorithms,” in *NeurIPS*, 2012, pp. 2951–2959.
- [5] N. Siegmund et al., “Performance-influence models for highly configurable systems,” in *Proc. ESEC/FSE*, ACM, 2015, pp. 284–294.
- [6] P. Jamshidi and G. Casale, “An uncertainty-aware approach to optimal configuration of stream processing systems,” in *Proc. MASCOTS*, 2016, pp. 39–48.
- [7] J. Gong and T. Chen, “Deep configuration performance learning: A systematic survey and taxonomy,” *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 1, art. 25, pp. 1–62, 2024.
- [8] F. Hutter, H. H. Hoos, and K. Leyton-Brown, “Sequential model-based optimization for general algorithm configuration,” in *Proc. LION*, 2011, pp. 507–523.
- [9] J. Bergstra et al., “Algorithms for hyper-parameter optimization,” in *NeurIPS*, 2011, pp. 2546–2554.
- [10] W. R. Thompson, “On the likelihood that one unknown probability exceeds another in view of the evidence of two samples,” *Biometrika*, vol. 25, no. 3/4, pp. 285–294, 1933.
- [11] D. Russo and B. Van Roy, “Learning to optimize via posterior sampling,” *Mathematics of Operations Research*, vol. 39, no. 4, pp. 1221–1243, 2014.
- [12] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [13] G. Ke et al., “LightGBM: A highly efficient gradient boosting decision tree,” in *NeurIPS*, 2017, pp. 3146–3154.
- [14] S. Falkner, A. Klein, and F. Hutter, “BOHB: Robust and efficient hyperparameter optimization at scale,” in *Proc. ICML*, 2018, pp. 1437–1446.
- [15] M. López-Ibáñez et al., “The irace package: Iterated racing for automatic algorithm configuration,” *Operations Research Perspectives*, vol. 3, pp. 43–58, 2016.
- [16] C. Oh, J. Tomczak, E. Gavves, and M. Welling, “Combinatorial Bayesian optimization using the graph Cartesian product,” in *NeurIPS*, 2019, pp. 2891–2901.
- [17] R. Baptista and M. Poloczek, “Bayesian optimization of combinatorial structures,” in *Proc. ICML*, 2018, pp. 462–471.
- [18] D. Van Aken et al., “Automatic database management system tuning through large-scale machine learning,” in *Proc. SIGMOD*, 2017, pp. 1009–1024.
- [19] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*, Addison-Wesley, 1989.
- [20] L. Kotthoff, “Algorithm selection for combinatorial search problems: A survey,” *AI Magazine*, vol. 35, no. 3, pp. 48–60, 2014.
- [21] V. N. Vapnik, *Statistical Learning Theory*, Wiley, 1998.
- [22] Y. Jin, “Surrogate-assisted evolutionary computation: Recent advances and future challenges,” *Swarm and Evolutionary Computation*, vol. 1, no. 2, pp. 61–70, 2011.

- [23] H. Ha and H. Zhang, “DeepPerf: Performance prediction for configurable software with deep sparse neural network,” in *Proc. ICSE*, 2019, pp. 1095–1106.
- [24] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., Springer, 2006.
- [25] E.-G. Talbi, *Metaheuristics: From Design to Implementation*, Wiley, 2009.
- [26] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas, “Taking the human out of the loop: A review of Bayesian optimization,” *Proc. IEEE*, vol. 104, no. 1, pp. 148–175, 2016.
- [27] Y. Bengio, A. Lodi, and A. Prouvost, “Machine learning for combinatorial optimization: A methodological tour d’horizon,” *European Journal of Operational Research*, vol. 290, no. 2, pp. 405–421, 2021.
- [28] O. Vinyals, M. Fortunato, and N. Jaitly, “Pointer Networks,” in *NeurIPS*, 2015, pp. 2692–2700.
- [29] W. Kool, H. van Hoof, and M. Welling, “Attention, Learn to Solve Routing Problems!” in *Proc. ICLR*, 2019.